Real-Time Industrial Defect Detection System

A computer vision pipeline for manufacturing quality control. A camera watches metal parts, a YOLOv8 model detects and locates surface defects, and the system reports each result to a PLC as PASS or REJECT. The service monitors itself with Prometheus and Grafana.

Overview

Domain	Computer vision, industrial quality control
Dataset	NEU-DET (NEU Metal Surface Defects) with bounding-box annotations
Defect classes	crazing, inclusion, patches, pitted_surface, rolled-in_scale, scratches
Model	YOLOv8n (Ultralytics)
Edge format	ONNX, run with ONNX Runtime
Service	FastAPI with Prometheus metrics
Deployment files	Dockerfile and Docker Compose (Prometheus and Grafana)

Results

Model: YOLOv8n, 50 epochs, 640 px, batch 16, trained on a Tesla T4 GPU (Google Colab) in about 0.9 hours. Scores are on the validation split (360 images). There is no separate test set.

Metric	Value
mAP50	0.727
mAP50-95	0.401
Precision	0.646
Recall	0.694
Speed on T4	about 2.5 ms inference per image

Class	mAP50	mAP50-95
patches	0.917	0.586
inclusion	0.842	0.480
pitted_surface	0.831	0.466
scratches	0.801	0.454
rolled-in_scale	0.545	0.244
crazing	0.427	0.177

Crazing is the weakest class (recall 0.295), so most crazing defects are missed. Detection is scored with mAP, precision and recall rather than a single accuracy figure, because an image can hold several defects and the box location matters.

What was built

Week 1: data pipeline

- Converted the NEU-DET annotations from VOC XML to YOLO format.
- Built an Albumentations pipeline (rotation, noise, lighting) with boxes transformed alongside the image. It produced 2,878 augmented image and label pairs.
- Combined the original and augmented data into the flat folders YOLO expects: 4,318 training images and 360 validation images.
- Wrote data checks: image-label integrity, class balance, empty labels and a box visualiser built on OpenCV.
- Wrote a custom evaluation framework (IoU, precision, recall).

Week 2: model training

- Trained YOLOv8n on the combined dataset on a Colab GPU.
- Compared training settings and reviewed per-class performance.

Week 3: edge optimization

- Exported the model to ONNX and benchmarked it with ONNX Runtime.
- Attempted TensorRT. It needs an NVIDIA GPU, so it could not run on our hardware.
- Wrote an OpenCV frame loop that reads a camera, video file or image folder, runs the model on each frame and draws boxes with an FPS counter.

Week 4: API, monitoring and deployment

- FastAPI service with POST /detect, GET /health and camera heartbeat endpoints. It rejects empty, oversized and non-image uploads.
- Each result is sent to a PLC as a UDP JSON message (PASS or REJECT plus box coordinates).
- Prometheus metrics for inference latency, defects per class, rejected requests and camera uptime.
- Dockerfile, Docker Compose, Prometheus and Grafana configuration.

Project structure

```
project_2/
  src/
    configs/data.yaml        YOLO dataset config
    data/                    conversion, augmentation, data checks, evaluation
    train/                   YOLOv8 training scripts
    edge/                    ONNX export, runtime check, frame inference, camera
client
    api/                     FastAPI service, mock PLC, API tests
  docs/                      HOW_TO_RUN.md, PROJECT_SUMMARY.md
  monitoring/                Prometheus config
  grafana/                   Grafana datasource config
  models/                    trained weights (not in git)
  data/                      dataset (not in git)
  Dockerfile
  docker-compose.yml
  requirements.txt
```

Quick start

```
git clone https://github.com/deepthi857/project_2.git
cd project_2
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

1. Place the NEU-DET dataset in `data/raw/NEU-DET/` (train and validation folders).
2. Run the data pipeline:

   ```
   python src\data\conert_annotations.py
   python src\data\augment.py
   python src\data\prepare_training_data.py
   python src\data\check_data_integrity.py
   ```

   The file name `conert_annotations.py` has a historical typo. Use it as is.
3. Train on a GPU (we used Google Colab), then copy the weights to `models/best.pt`:

```
python src\train\train_model.py
```

4. Start the API:

   ```
   uvicorn src.api.main:app --reload
   ```

   Open http://127.0.0.1:8000/docs to test it, and http://127.0.0.1:8000/metrics for Prometheus metrics.

Without `models/best.pt` the API falls back to the generic `yolov8n.pt`, which does not detect these defects.

PLC and camera demo

Open three terminals, each with the virtual environment active:

```
python src\api\mock_plc.py
uvicorn src.api.main:app --reload
python src\edge\camera_client.py
```

The mock PLC prints a PASS or REJECT line for every frame. `http://127.0.0.1:8000/camera/status` shows whether the camera is up.

Docker

```
docker compose up --build
```

This starts the API, Prometheus (http://localhost:9090) and Grafana (http://localhost:3000). The Docker files are in the repository; they have not been built on the development laptop.

Known limitations

- The development laptop's CPU lacks AVX instructions, so local training crashed with an Illegal Instruction error. Training was done on Google Colab.
- TensorRT needs an NVIDIA GPU, which was not available. ONNX Runtime is used instead.
- On the CPU-only laptop the ONNX inference loop runs at about 1.4 FPS. The T4 figure above is the model's speed on a GPU.
- The PLC link is a UDP JSON stand-in. A real line would replace `send_to_plc()` with its own protocol, such as Modbus TCP or OPC UA.
- Scores are on the validation split with no held-out test set.

Next steps

- Run on a GPU edge device with TensorRT.
- Connect a real PLC protocol.
- Add crazing-focused data and augmentation.
- Hold out a separate test set.

Team

- S Deepthi Reddy
- G. Sai Maitra
- M. Shashank
- Shreyansh Gupta

More documentation

- `docs/HOW_TO_RUN.md` : full setup, run and troubleshooting steps
- `docs/PROJECT_SUMMARY.md` : results, status of each component and limitations